

CLAP Parameters

The `clap.params` extension

Gabriel

CCRMA Workshop 2026 — Building Audio Plugins from First Principles

CLAP

CLAP is an application binary interface. Any programming language can implement CLAP (as a host or a plugin). But CLAP is written in C. How does this work?

Compiled C code is very predictable. It's easy for any language to invoke compiled C code if it knows “what to expect”.

Implementing CLAP = giving CLAP structs full of function pointers.

CLAP Helpers

If you are writing your plugin in C++, you might use the `clap-helpers` repository.

This “wraps” some of the low level C++ functionality in a format more familiar to object-oriented programmers, where you override virtual methods from a base class.

The function signatures are still roughly the same, though.

An extension is a string and a struct

```
// ext/log.h -- a complete extension:

static const char CLAP_EXT_LOG[] = "clap.log";

typedef struct clap_host_log {
    // Log a message through the host.
    // [thread-safe]
    void(CLAP_ABI *log)(const clap_host_t *host,
                        clap_log_severity severity,
                        const char *msg);
} clap_host_log_t;
```

An extension is defined **entirely in its header**: a string id plus couple structs of function pointers.

The string identifies the extension you're implementing.

The function pointers point to the various functions that actually do the implementing.

The get_extension Function

```
static const void *my_plug_get_extension(  
    const clap_plugin_t *plugin,  
    const char *id) {  
    if (!strcmp(id, CLAP_EXT_AUDIO_PORTS))  
        // "Here are my audio ports functions!"  
        return &s_my_plug_audio_ports;  
    if (!strcmp(id, CLAP_EXT_PARAMS))  
        // "Here are my params functions!"  
        return &s_my_plug_params;  
    return NULL; // "I don't implement that"  
}
```

```
// the mirror image: in init() the plugin  
// queries the host the same way  
plug->host_log = (const clap_host_log_t *)  
    plug->host->get_extension(plug->host,  
                             CLAP_EXT_LOG);
```

This is the **entire registration system**.

CLAP asks your plugin if you implement a extension matching a given string.

You return a struct containing pointers to all the extension functions.

Implementing extensions

```
// ordinary static functions...
static uint32_t my_plug_audio_ports_count(
    const clap_plugin_t *plugin,
    bool is_input) {
    return 1;
}

// clap_plugin_audio_ports_t is defined by CLAP,
// the rest is us
static const clap_plugin_audio_ports_t
my_audio_ports_struct = {
    .count = my_plug_audio_ports_count,
    .get    = my_plug_audio_ports_get,
};
```

To “implement” an extension, we fill the extension struct with function pointers that point to our various functions.

The host will then call these at will.

Everything is static!!

```
// this is our custom struct! It isn't a CLAP type.
typedef struct {
    clap_plugin_t    plugin;
    const clap_host_t *host;
    float            gain;    // our state
    float            someOtherParam; // more state
    ... // cont'd
} my_plug_t;

clap_plugin_t *my_plug_create(
    const clap_host_t *host) {
    // allocate a plugin structure
    my_plug_t *p = calloc(1, sizeof(*p));
    // store a pointer to our custom struct
    // inside CLAP's designated data variable
    p->plugin.plugin_data = p;
    // ...fill in the function pointers...
    return &p->plugin;
}

// first line of every callback:
my_plug_t *plug = plugin->plugin_data;
```

The extension functions are **static**. They're shared by **every instance of the plugin**. They can't hold per-instance state.

Luckily CLAP's `clap_plugin_t` has a variable called `plugin_data` which can store a pointer to anything.

`create` allocates our custom struct and stores its address in `plugin_data`.

Every extension function receives the `clap_plugin_t *` as its first argument, which contains `plugin_data`.

Who calls who?

The plugin can also query the host for extension support, and call its extension functions, which are all exposed to us via a “host” struct.

What is a parameter?

- ▶ A parameter is a **double** that the host can see and automate: Gain, Cutoff, Mix, ...
- ▶ Every parameter has a **stable id** chosen by the plugin. The id never changes.
- ▶ **The plugin owns the parameter value.** The host acts as a controller on top of the plugin's parameters.

It's an extension

```
// the host requests the extension by name
static const void *my_plug_get_extension(
    const clap_plugin_t *plugin,
    const char *id) {
    if (!strcmp(id, CLAP_EXT_PARAMS))
        return &s_my_plug_params;
    ...
}

// ...and gets a table of function pointers:
static const clap_plugin_params_t
s_my_plug_params = {
    .count          = my_plug_params_count,
    .get_info       = my_plug_params_info,
    .get_value      = my_plug_params_value,
    .value_to_text  = my_plug_params_value_to_text,
    .text_to_value  = my_plug_params_text_to_value,
    .flush          = my_plug_params_flush,
};
```

The host asks for "clap.params" and receives a struct of function pointers.

These **six functions** are the whole plugin side of the parameter system. The host will call these plugins.

The host has a host-side extension, `clap_host_params`, with **three functions** the plugin can call.

We fetch it with `host->get_extension` in `init`.

The count Function

```
static uint32_t my_params_count(  
    const clap_plugin_t *plugin) {  
    return 1;    // just "Gain" for now  
}
```

The simplest hook: how many parameters exist. [main-thread]

The host then iterates indices 0 .. count-1 and calls `get_info` on each one.

The get_info Function

```
static bool my_params_info(  
    const clap_plugin_t *plugin,  
    uint32_t param_index,  
    clap_param_info_t *info) {  
    if (param_index == 0) {    // "Gain"  
        info->id      = 0; // fixed forever; don't change  
        info->flags = CLAP_PARAM_IS_AUTOMATABLE;  
        strncpy(info->name, "Gain",  
                CLAP_NAME_SIZE - 1);  
        info->min_value      = -5.0; // plain values,  
        info->max_value      =  5.0; // not 0-1 norm.  
        info->default_value =  2.0;  
        return true;  
    }  
    if (param_index == 1) {    // next parameter  
        ...                    // id 1, name, range  
        return true;  
    }  
    return false;  
}
```

Called by the host after the `count` function is called, once per parameter.

Note that we don't **return** the info struct.

CLAP hands us a pointer to the struct, and we fill out its info. We return a success/failure flag instead.

The get_value Function

```
static bool my_params_value(  
    const clap_plugin_t *plugin,  
    clap_id param_id,  
    double *out_value) {  
    my_plug_t *plug = plugin->plugin_data;  
    if (param_id == 0) {    // "Gain"  
        *out_value = plug->gain;  
        return true;  
    }  
    if (param_id == 1) {    // next parameter  
        ...  
        return true;  
    }  
    return false;  
}
```

Called by the host when it wants to know the value of a parameter.

Because **the plugin owns the value**, this is the only way the host learns it. The host doesn't store it.

Addressed by **id**, not `get_info` index.

The value_to_text Function

```
static bool my_params_value_to_text(  
    const clap_plugin_t *plugin,  
    clap_id param_id, double value,  
    char *display, uint32_t size) {  
    int n = -1;  
    if (param_id == 0)        // "Gain"  
        n = snprintf(display, size, "%.2f", value);  
    if (param_id == 1)        // next parameter  
        n = ...;              // format param 1 as text  
    return n >= 0 && (uint32_t)n < size;  
}
```

Turns a value into the string the user sees: "0.50", "-6.0 dB", "2.3 kHz".

The text_to_value Function

```
static bool my_params_text_to_value(  
    const clap_plugin_t *plugin,  
    clap_id param_id,  
    const char *display, double *out_value) {  
    char *end = NULL;  
    double v = strtod(display, &end);  
    if (end == display)  
        return false;    // no digits parsed  
    if (param_id == 0) {    // "Gain"  
        *out_value = v;  
        return true;  
    }  
    if (param_id == 1) {    // next parameter  
        ...  
        return true;  
    }  
    return false;  
}
```

The inverse: the user **types** "0.25" (or "-12 dB") into the DAW and the plugin parses it back into a plain value.

Return **false** on text you can't parse.

Together with `value_to_text`, this pair defines how your parameter reads and writes as text.

Where is `set_value`?

Gotcha! There isn't one.

- ▶ Parameter changes travel as **events**, like MIDI does
- ▶ Each event is **timestamped** with a sample offset inside the current block.
- ▶ There are two delivery channels:
 - ▶ during `process()` — alongside audio, sample-accurate;
 - ▶ during `flush()` — when no audio is running.
- ▶ Events flow **both ways**: host → plugin (automation, DAW knobs) and plugin → host (edits made in the plugin's own GUI).

Receiving CLAP_EVENT_PARAM_VALUE

```
static void my_process_event(  
    my_plug_t *plug,  
    const clap_event_header_t *hdr) {  
    switch (hdr->type) {  
        case CLAP_EVENT_PARAM_VALUE: {  
            const clap_event_param_value_t *ev =  
                (const clap_event_param_value_t *)hdr;  
            if (ev->param_id == 0)    // "Gain"  
                plug->gain = (float)ev->value;  
            if (ev->param_id == 1)    // next parameter  
                ... ;  
            break;  
        }  
    }  
}
```

During audio playback, the host signals a parameter change as an event, alongside other events like MIDI.

It's up to you to intercept and process it.

Plugin → host: output events and gestures

```
// suppose the user grabs a knob
// (this code would be either in flush or process)
clap_event_param_gesture_t g = {
    .header = {
        .size = sizeof(g),
        .time = 0,
        .space_id = CLAP_CORE_EVENT_SPACE_ID,
        .type = CLAP_EVENT_PARAM_GESTURE_BEGIN,
    },
    .param_id = 0,
};
out->try_push(out, &g.header);

// ...then, later: one PARAM_VALUE event per change...
// ...then, later: PARAM_GESTURE_END when they let go
```

When the *plugin* changes a value, it must tell the host.

GESTURE_BEGIN / GESTURE_END
sandwich the edit so the host
records it as one automation gesture
and can undo it as one step.

The flush Function

```
static void my_plug_params_flush(  
    const clap_plugin_t *plugin,  
    const clap_input_events_t *in,  
    const clap_output_events_t *out) {  
    my_plug_t *plug = plugin->plugin_data;  
    const uint32_t n = in->size(in);  
    for (uint32_t i = 0; i < n; ++i)  
        my_plug_process_event(  
            plug, in->get(in, i));  
}
```

What if a parameter is changed when audio isn't changing?

This weird little guy is used for parameter updates when audio isn't playback. The host calls it when it needs a parameter sync but isn't running `process()`.

The request_flush Function (host)

```
// in init(): the host's half of clap.params
plug->host_params =
    plug->host->get_extension(plug->host,
                             CLAP_EXT_PARAMS);

// GUI thread: knob moved, but audio is stopped
if (plug->host_params)
    plug->host_params->request_flush(plug->host);
// the host will soon call process() or flush()
```

But what if audio is not playing and the plugin needs to change a parameter?

The plugin *requests* a flush from the host by invoking the host's extension function.

The host calls the plugin's flush function shortly afterward.